ML Lab: Boosting

Amlan Chakrabarti

Reducing bias and decreasing variance

- **Bagging reduces variance by averaging**
- **Bagging has little effect on bias**
- **Can we average and reduce bias?**
- **Solution: Boosting**

Boosting

- **An iterative procedure to adaptively change distribution of training data by focusing more on previously misclassified records**
- **Initially, all N records are assigned equal weights**
- **Unlike bagging, weights may change at the end of a boosting round**

Boosting: Overall Approach

1. Equal weights are assigned to each training instance (1/N for round 1) at first round
2. After a classifier C_i is learned, the weights are adjusted to allow the subsequent classifier
3. C_{i+1} to "pay more attention" to data that were misclassified by C_i.
4. Final boosted classifier C* combines the votes of each individual classifier
5. Weight of each classifier's vote is a function of its accuracy

Adaboost – popular boosting algorithm

Adaboost Algorithm

- given training set $(x_1, y_1), \ldots, (x_m, y_m)$
- $y_i \in \{-1, +1\}$ correct label of instance $x_i \in X$
- for $t = 1, \ldots, T$:
  - construct distribution D_t on $\{1, \ldots, m\}$
  - find weak classifier ("rule of thumb")

$$h_t : X \to \{-1, +1\}$$

with error ϵ_t on D_t:

$$\epsilon_t = \Pr_{i \sim D_t}[h_t(x_i) \neq y_i]$$

- output final/combined classifier H_{final}

Adaboost

- constructing D_t:
  - $D_1(i) = 1/m$
  - given D_t and h_t:

$$D_{t+1}(i) = \frac{D_t(i)}{Z_t} \times \begin{cases} e^{-\alpha_t} & \text{if } y_i = h_t(x_i) \\ e^{\alpha_t} & \text{if } y_i \neq h_t(x_i) \end{cases}$$

$$= \frac{D_t(i)}{Z_t} \exp(-\alpha_t \, y_i \, h_t(x_i))$$

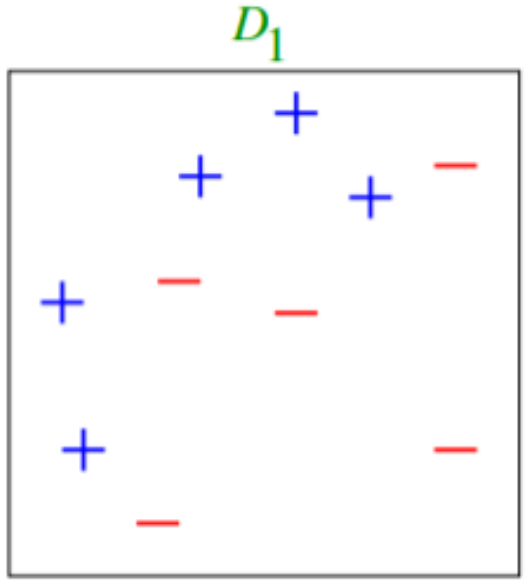
where Z_t = normalization factor

$$\alpha_t = \frac{1}{2} \ln\left(\frac{1-\epsilon_t}{\epsilon_t}\right) > 0$$

$$Z_t = \sum_{i=1}^{m} D_t(i)\, e^{-\alpha_t y_i h_t(x_i)}$$

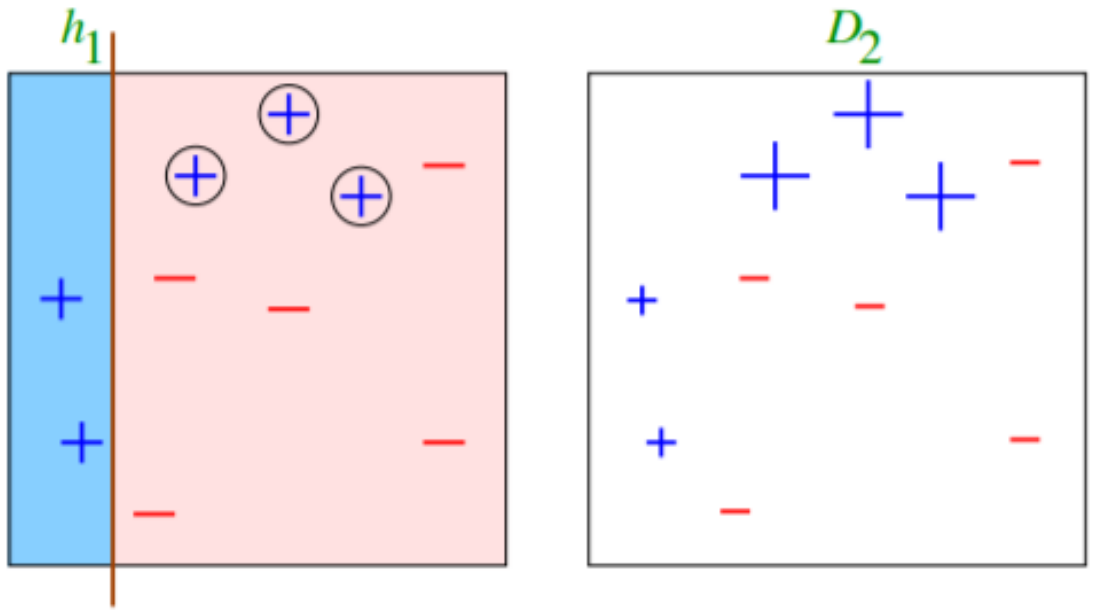
- final classifier:
  - $H_{\text{final}}(x) = \text{sign}\left(\sum_t \alpha_t h_t(x)\right)$

Adaboost: Example

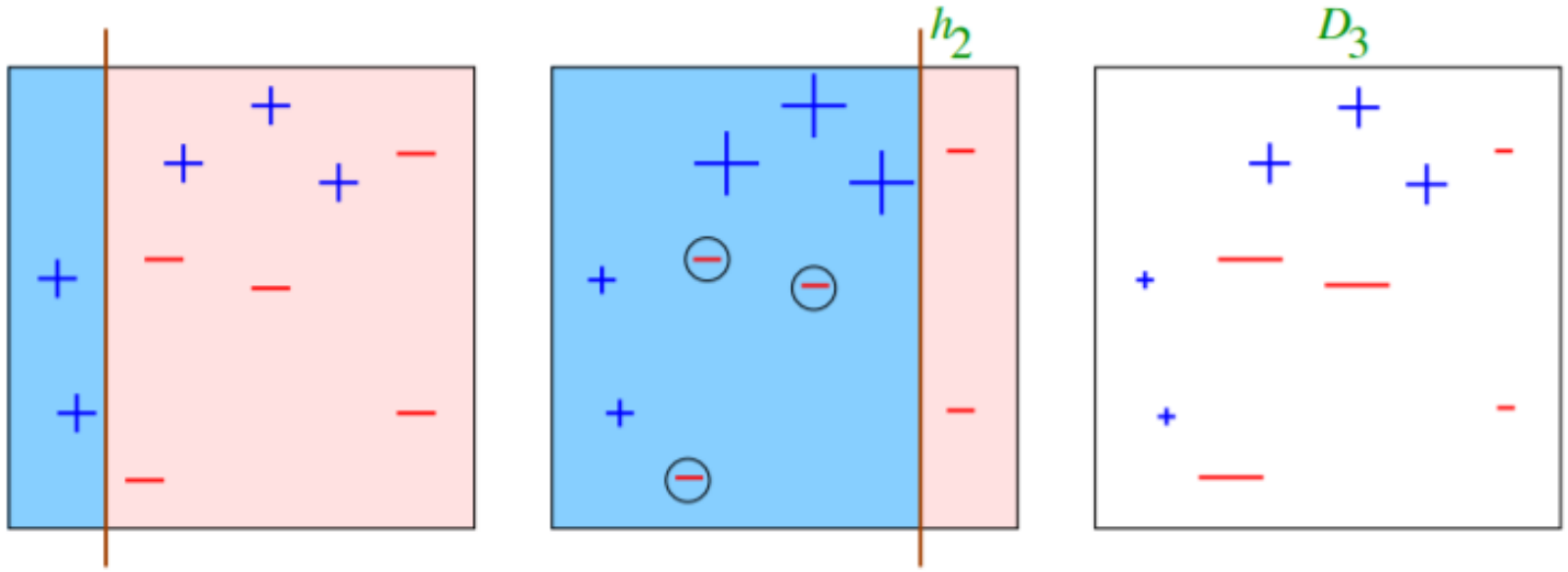


Weak classifiers: Horizontal or vertical half planes

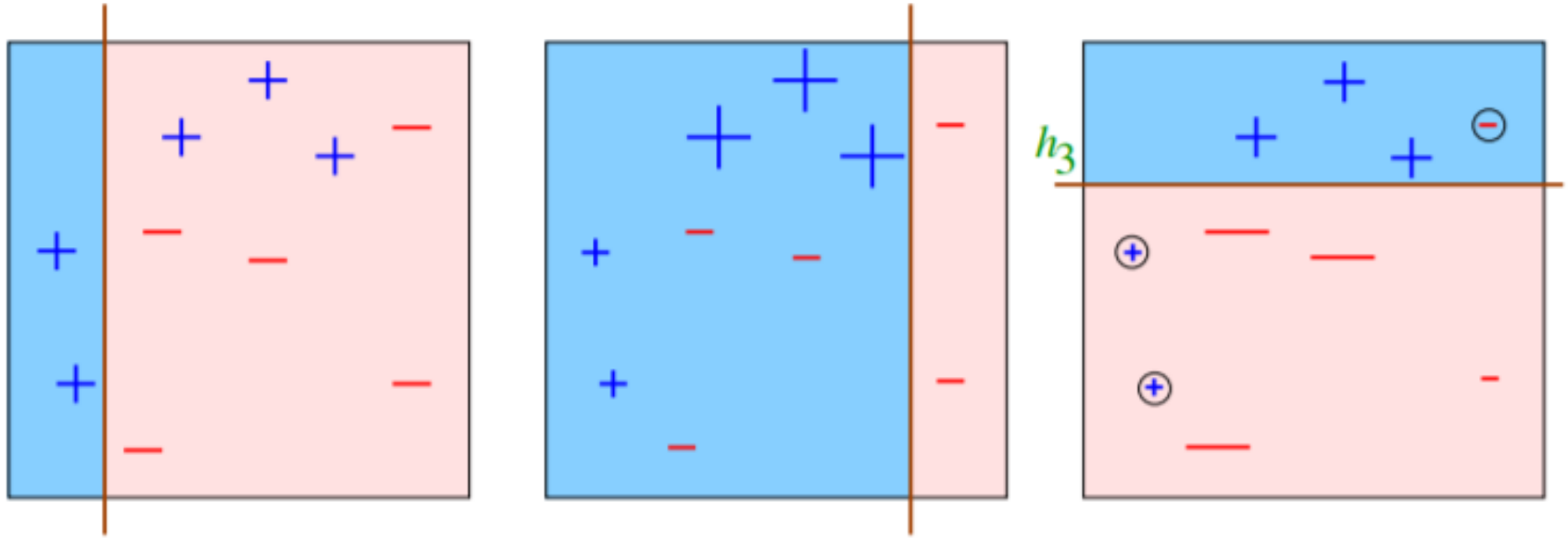
Example: Round 1



Example: Round 2



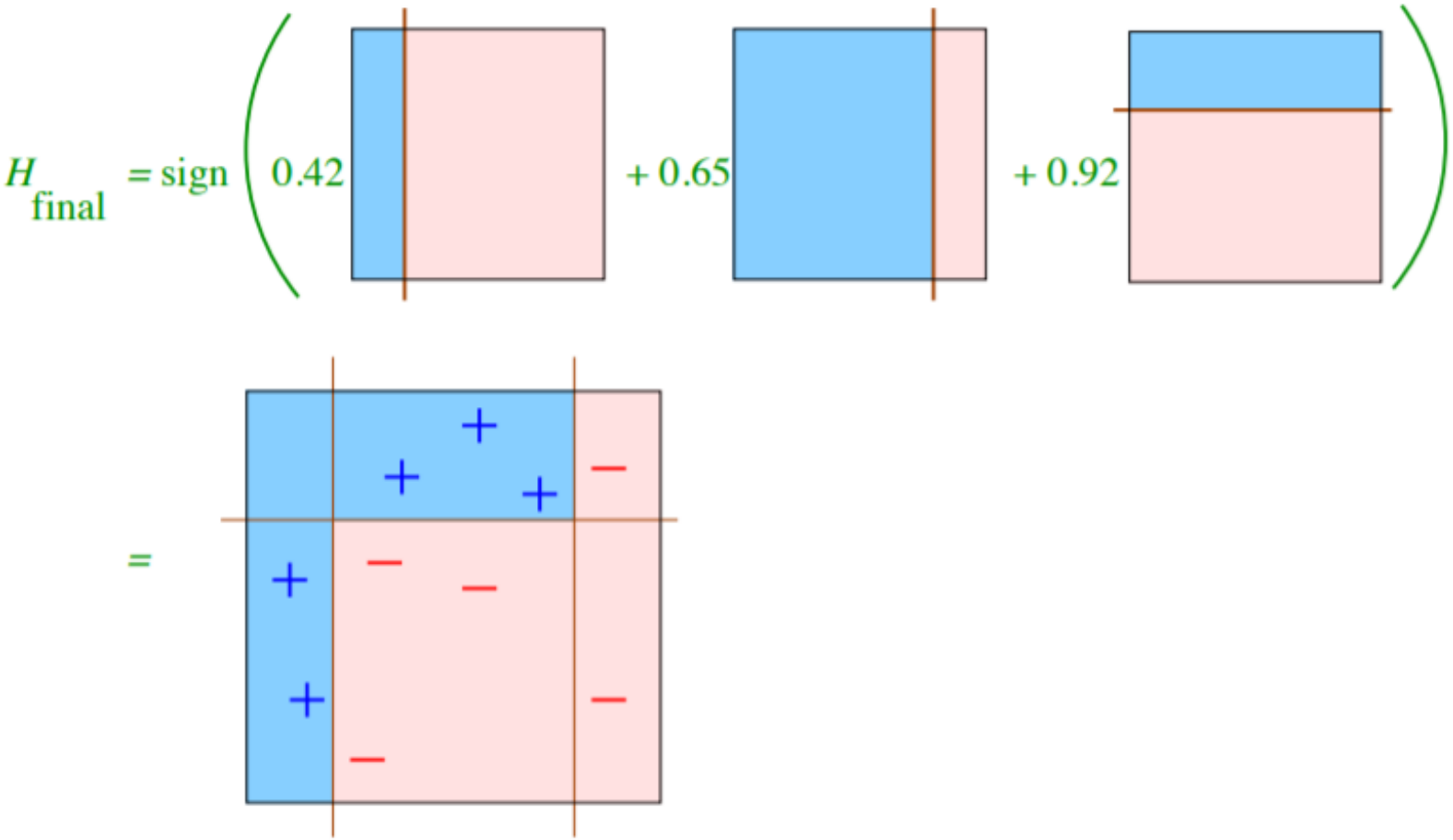
Example: Round 3



$\varepsilon_3 = 0.14$

$\alpha_3 = 0.92$

Example: Final Classifier



Adaboost: Voted Classifier

- **Adaboost tries to combine many weak classifiers**

- **Considers voted combinations of simple binary (+/-) classifiers**

$$h_m(\mathbf{x}) = \alpha_1 h(\mathbf{x}; \theta_1) + \ldots + \alpha_m h(\mathbf{x}; \theta_m)$$

the weights α_i (≥ 0) emphasize the component classifier that is more reliable than the others

Loss Function

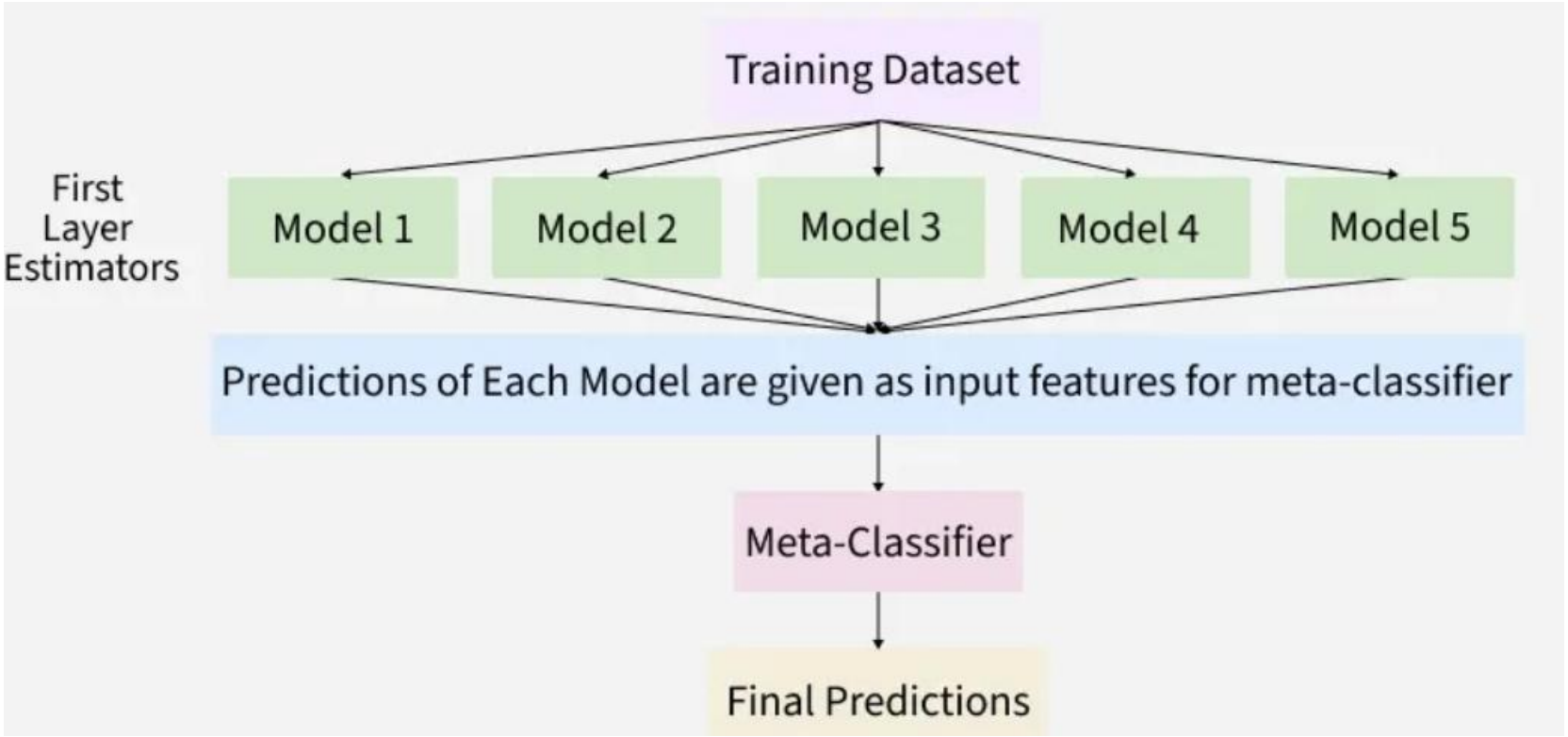
- **Simple exponential loss could be used**

$$\exp\{ -y\, h_m(\mathbf{x}) \}$$ **y is the target label**

where $h_m(\mathbf{x}) = \alpha_1\, h(\mathbf{x}; \theta_1) + \ldots + \alpha_m\, h(\mathbf{x}; \theta_m)$

Stacking

- **Stacking is a ensemble learning technique where the final model known as the “stacked model" combines the predictions from multiple base models.**
- **The goal is to create a stronger model by using different models and combining them.**



Big Picture: What is Stacking?

- **Stacking is an ensemble learning technique where:**
  - We train multiple different models (called base learners or level-0 models)
  - Then we train another model (called the meta-learner or level-1 model)
  - The meta-learner learns how to best combine the predictions of the base models
  - Instead of averaging or voting, stacking learns the optimal way to combine models.

Working of Stacking

- **Start with training data:** We begin with the usual training data that contains both input features and the target output.
- **Train base models:** The base models are trained on this training data. Each model tries to make predictions based on what it learns.
- **Generate predictions:** After training the base models make predictions on new data called validation data or out-of-fold data. These predictions are collected.
- **Train meta-model:** The meta-model is trained using the predictions from the base models as new features. The target output stays the same and the meta-model learns how to combine the base model predictions.
- **Final prediction:** When testing the base models make predictions on new, unseen data. These predictions are passed to the meta-model which then gives the final prediction.

Architecture of a Stacking Ensemble

Level 0: Base Models

These are trained on the original dataset.

Examples:

- Logistic Regression
- Random Forest
- Gradient Boosting
- SVM
- Neural Network

Level 1: Meta Model

This model takes **predictions from base models as inputs**.

Examples:

- Linear Regression / Logistic Regression
- XGBoost
- LightGBM
- Shallow neural network

Out of Fold Prediction

Assume:

- 5-fold cross-validation
- One base model

Step 1: Split Data into 5 Folds

```
Fold 1 | Fold 2 | Fold 3 | Fold 4 | Fold 5
```

Step 2: Train & Predict (Repeat 5 Times)

Iteration	Train On	Predict On
1	F2–F5	F1
2	F1,F3–F5	F2
3	F1,F2,F4,F5	F3
4	F1–F3,F5	F4
5	F1–F4	F5

Each data point gets predicted **once**, by a model that never saw it.

Step 3: Collect OOF Predictions

You end up with:

- One prediction per training sample
- All predictions are out-of-fold

How Meta-Learner learns?

- **Meta-learner is trained on:**
  - X = OOF predictions
  - y = true labels

Bias–Variance Perspective

Component	Role
Base models	Diverse biases & errors
Meta-learner	Reduces variance and bias by combining them

Meta-learner acts like an **error corrector**.

Advantages of Stacking

- **Better Performance:** Stacking often results in higher accuracy by combining predictions from multiple models making the final output more reliable.
- **Combines Different Models:** It allow to use various types of models like decision trees, logistic regression, SVM made from each model's unique strengths.
- **Reduces Overfitting:** When implemented with proper cross-validation it can reduce the risk of overfitting by balancing out the weaknesses of individual models.
- **Learns from Mistakes:** The meta-model is trained to recognize where base models go wrong and improves the final prediction by correcting those errors.
- **Customizable:** We can choose any combination of base and meta-models depending on our dataset and problem type making it highly flexible.

Limitations of Stacking

- **Complex to Implement:** Compared to simple models or even bagging or boosting, stacking requires more steps and careful setup.
- **Slow Training Time:** Since you're training multiple models plus a meta-model it can be slow and computationally expensive.
- **Hard to Interpret:** With multiple layers of models it becomes difficult to explain how the final prediction was made.
- **Risk of Overfitting:** If the meta-model is too complex or if there's data leakage it can overfit the training data.
- **Needs More Data:** It performs better when you have enough data, especially for training both base and meta-models effectively